

第12章 数据存储与管理

人工智能学院
古 晶

西安电子科技大学



第12章 数据存储与管理

- 12.1 传统的数据存储与管理技术
- 12.2 大数据时代的数据存储与管理技术
- 12.3 大数据处理架构Hadoop
- 12.4 分布式文件系统HDFS
- 12.5 NoSQL数据库
- 12.6 云数据库
- 12.7 分布式数据库HBase
- 12.8 知识点小结



第12章 数据存储与管理

□12.1 传统的数据存储与管理技术

□12.2 大数据时代的数据存储与管理技术

□12.3 大数据处理架构Hadoop

□12.4 分布式文件系统HDFS

□12.5 NoSQL数据库

□12.6 云数据库

□12.7 分布式数据库HBase

□12.8 知识点小结



12.1.1 文件系统



- 文件系统是操作系统用于明确存储设备（常见的是磁盘，也有基于NAND Flash的固态硬盘）或分区上的文件的方法和数据结构，即在存储设备上组织文件的方法。操作系统中负责管理和存储文件信息的软件机构称为**文件管理系统**，简称“**文件系统**”。
- 我们平时在计算机上使用的WORD文件、PPT文件、文本文件、音频文件、视频文件等，都是由操作系统中的**文件系统**进行统一管理的。



12.1.2 关系数据库

总体而言，关系数据库具有如下**特点**：

(1) **存储方式**。关系数据库采用**表格**的储存方式，数据以行和列的方式进行存储，要读取和查询都十分方便。

(2) **存储结构**。关系数据库按照**结构化**的方法存储数据，每个数据表的结构都必须**事先定义好**（比如表的名称、字段名称、字段类型、约束等），然后再根据表的结构存入数据，这样做的好处就是，由于数据的形式和内容在存入数据之前就已经定义好了，所以，整个数据表的可靠性和稳定性都比较高，但是带来的问题就是，数据模型不够灵活，一旦存入数据后，如果需要修改数据表的结构就会十分困难。

(3) **存储规范**。关系数据库为了规范化数据、减少重复数据以及充分利用好存储空间，把数据按照**最小关系表**的形式进行存储，这样数据管理就可以变得很清晰、一目了然。当存在多个表时，表和表之间通过**主外键关系**发生关联，并通过**连接查询**获得相关结果。

(4) **扩展方式**。由于关系数据库将数据存储和数据表中，数据操作的**瓶颈**出现在多张数据表的操作中，而且数据表越多这个问题越严重。如果要缓解这个问题，只能提高处理能力，也就是选择速度更快、性能更高的计算机，这样的方法虽然具有一定的拓展空间，但是这样的拓展空间是非常有限的，也就是一般的关系型数据库只具备有限的纵向扩展能力。



12.1.2 关系数据库



(5) **查询方式**。关系数据库采用**结构化查询语言**（即SQL: Structured Query Language）来对数据库进行查询。结构化查询语言是高级的非过程化编程语言，允许用户在高层数据结构上工作。它不要求用户指定对数据的存放方法，也不需要用户了解具体的数据存放方式，所以，各种具有完全不同底层结构的数据库系统，可以使用**相同**的结构化查询语言作为数据输入与管理的接口。结构化查询语言语句可以嵌套，这使它具有极大的灵活性和强大的功能。

(6) **事务性**。关系数据库可以支持事务的**ACID特性**（原子性（Atomicity）、一致性（Consistency）、隔离性（Isolation）、持久性（Durability））。当事务被提交给了DBMS（数据库管理系统），则DBMS需要确保该事务中的所有操作都成功完成且其结果被永久保存在数据库中，如果事务中有的操作没有成功完成，则事务中的所有操作都需要被回滚，回到事务执行前的状态，从而确保数据库状态的一致性。

(7) **连接方式**。不同的关系数据库产品都遵守一个统一的数据库连接接口标准，即**ODBC**（Open Database Connectivity）。ODBC的一个显著优点是，用它生成的程序是**与具体的数据库产品无关**的，这样可以为数据库用户和开发人员屏蔽不同数据库异构环境的复杂性。**ODBC**提供了数据库访问的统一接口，为应用程序实现与平台的**无关性**和**可移植性**提供了基础，因而获得了广泛的支持和应用。



12.1.3 数据仓库



数据仓库 (Data Warehouse) 是一个面向主题的、集成的、相对稳定的、反映历史变化的数据集合，用于支持管理决策：

(1) **面向主题**。操作型数据库的数据组织面向事务处理任务，而数据仓库中的数据是按照一定的主题域进行组织。主题是指用户使用数据仓库进行决策时所关心的重点方面，一个主题通常与多个操作型信息系统相关。

(2) **集成**。数据仓库的数据来自于分散的操作型数据，将所需数据从原来的数据中抽取出来，进行加工与集成、统一与综合之后才能进入数据仓库。

(3) **相对稳定**。数据仓库是不可更新的，数据仓库主要是为决策分析提供数据，所涉及的操作主要是数据的查询。

(4) **反映历史变化**。在构建数据仓库时，会每隔一定的时间（比如每周、每天或每小时）从数据源抽取数据并加载到数据仓库。



12.1.4 并行数据库



- **并行数据库**是指那些在无共享的体系结构中进行数据操作的数据库系统。
- 这些系统大部分采用了关系数据模型并且支持SQL语句查询，但为了能够并行执行SQL的查询操作，系统中采用了两个关键技术：**关系表的水平划分**和**SQL查询的分区执行**。
- 并行数据库系统的**目标**是**高性能**和**高可用性**，通过多个节点并行执行数据库任务，提高整个数据库系统的性能和可用性。



12.1.4 并行数据库



并行数据库系统的主要缺点就是**没有较好的弹性**，而这种特性对中小型企业 and 初创企业是有利的。人们在对并行数据库进行设计和优化的时候认为**集群中节点的数量是固定**的，若需要对集群进行扩展和收缩，则必须为数据转移过程制订周全的计划。这种数据转移的代价是昂贵的，并且会导致系统在某段时间内不可访问，而这种较差的灵活性直接影响到并行数据库的弹性以及现用现付商业模式的实用性。

并行数据库的另一个问题就是**系统的容错性较差**，过去人们认为**节点故障**是个特例，并不经常出现，因此系统只提供事务级别的容错功能，如果在查询过程中节点发生故障，那么整个查询都要从头开始重新执行。这种重启任务的策略使得并行数据库难以在拥有数以千个节点的集群上处理较长的查询，因为在这类集群中节点的故障经常发生。基于这种分析，**并行数据库只适合于资源需求相对固定的应用程序**。不管怎样，并行数据库的许多设计原则为其他海量数据系统的设计和优化提供了比较好的借鉴。



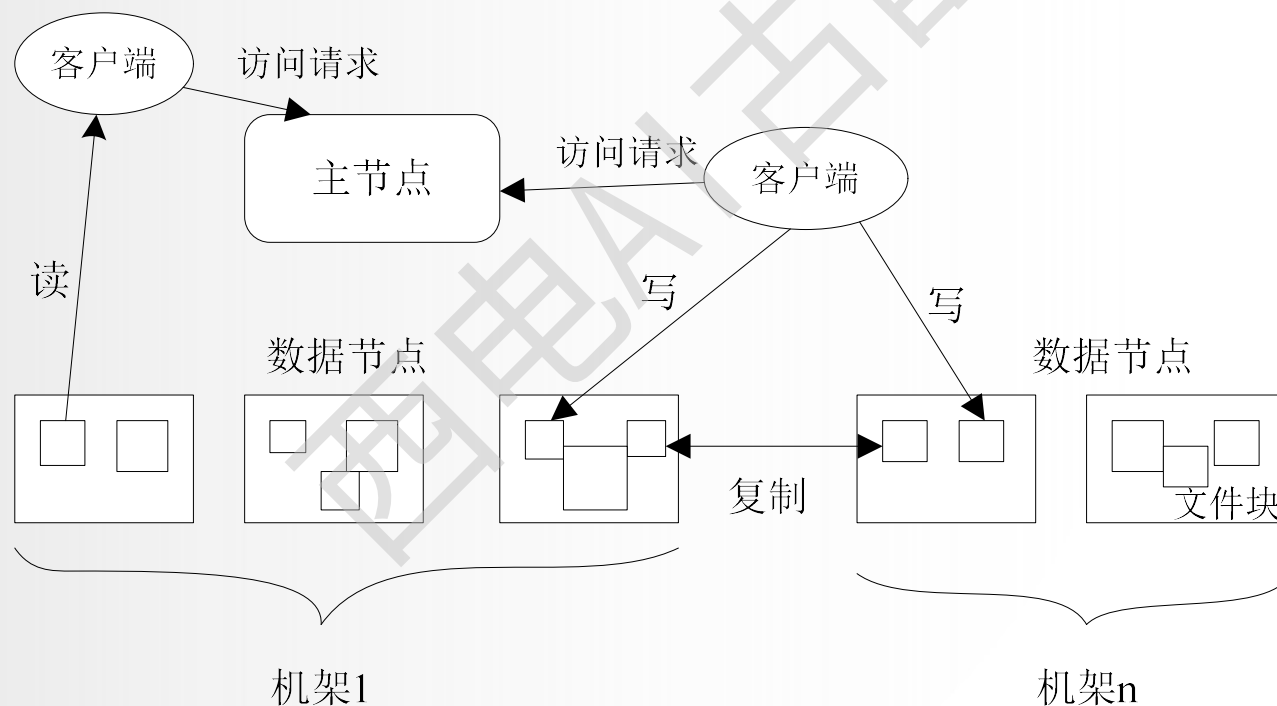
第12章 数据存储与管理

- 12.1 传统的数据存储与管理技术
- 12.2 大数据时代的数据存储与管理技术**
- 12.3 大数据处理架构Hadoop
- 12.4 分布式文件系统HDFS
- 12.5 NoSQL数据库
- 12.6 云数据库
- 12.7 分布式数据库HBase
- 12.8 知识点小结



12.2.1 分布式文件系统

分布式文件系统 (Distributed File System) 是一种通过**网络**实现文件在**多台主机**上进行分布式存储的文件系统。



分布式文件系统的整体结构



12.2.2 NewSQL和NoSQL数据库



(1) NewSQL数据库

- NewSQL是对各种新的可扩展、高性能数据库的**简称**，这类数据库不仅具有对海量数据的存储管理能力，还保持了传统数据库支持**ACID**和**SQL**等特性。
- 目前具有代表性的**NewSQL数据库**主要包括Spanner、Clustrix、GenieDB、ScalArc、Schooner、VoltDB、RethinkDB、ScaleDB、Akiban、CodeFutures、ScaleBase、Translattice、NimbusDB、Drizzle、Tokutek、JustOne DB等。



12.2.2 NewSQL和NoSQL数据库



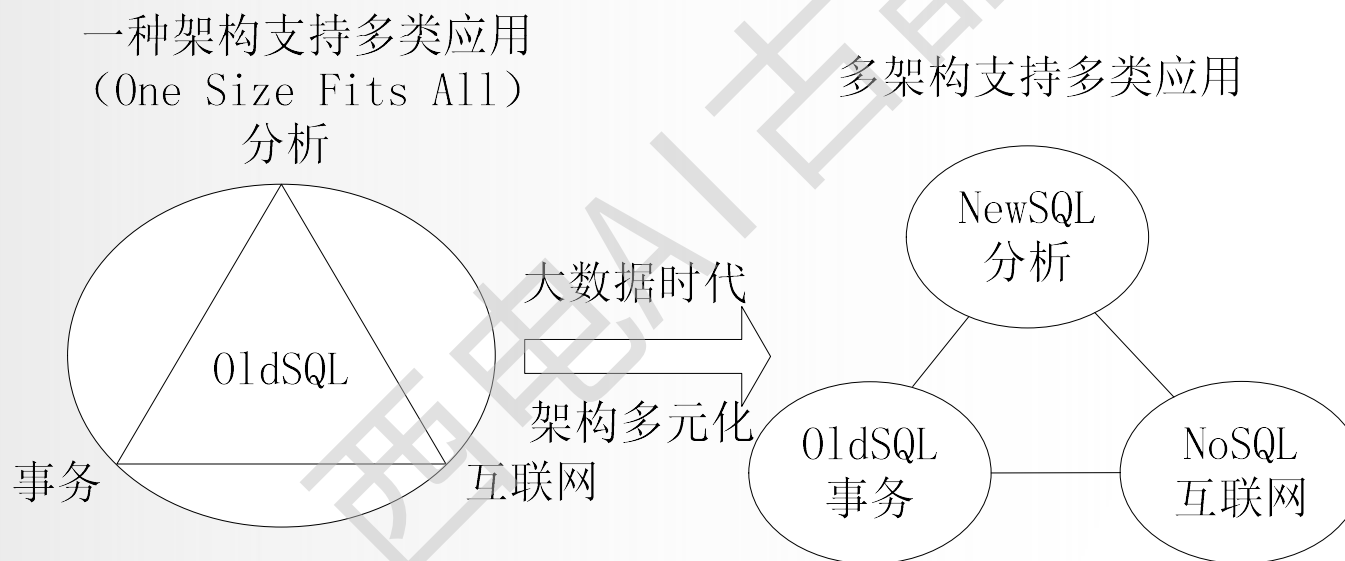
(2) NoSQL数据库

- NoSQL是一种不同于关系数据库的数据库管理系统设计方式，是对**非关系型数据库**的**统称**，它所采用的数据模型并非传统关系数据库的关系模型，而是类似键/值、列族、文档等非关系模型。
- NoSQL数据库**没有固定的表结构**，通常也不存在连接操作，也没有严格遵守ACID约束，因此，与关系数据库相比，NoSQL具有灵活的水平可扩展性，可以支持海量数据存储。



12.2.2 NewSQL和NoSQL数据库

(3) 大数据引发数据库架构变革



大数据引发数据库架构变革



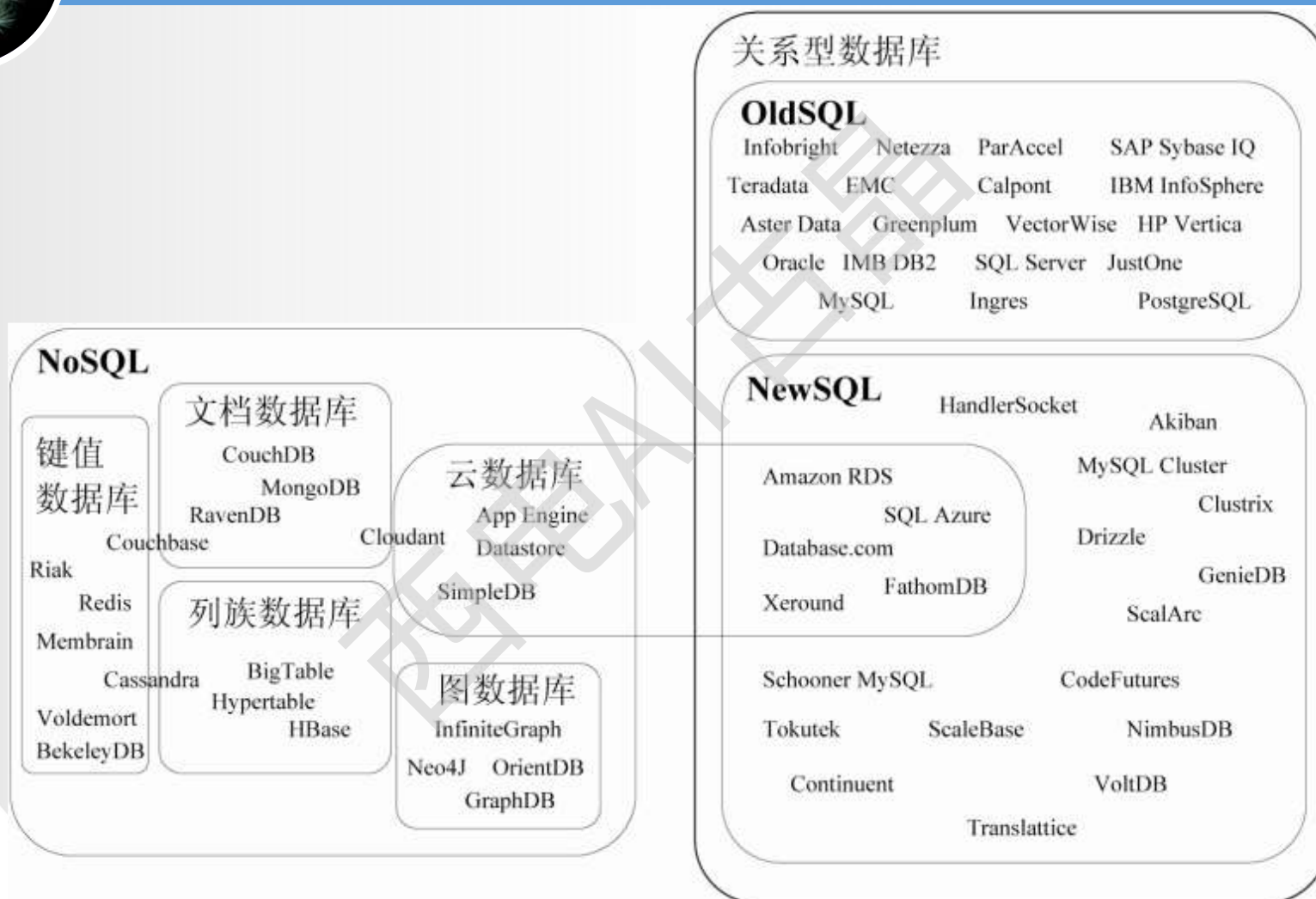
12.2.3 云数据库



- 研究机构IDC预言，大数据将按照每年**60%**的速度增加，其中包含**结构化**和**非结构化**数据。如何方便、快捷、低成本地存储这些海量数据，是许多企业和机构面临的一个严峻挑战。
- **云数据库**就是一个非常好的解决方案，目前云服务提供商正通过云技术推出更多可在**公有云中托管数据库**的方法，将用户从繁琐的数据库硬件定制中解放出来，同时让用户拥有强大的**数据库扩展能力**，满足海量数据的存储需求。
- 此外，云数据库还能够很好地满足企业**动态变化**的数据存储需求和中小企业的低成本数据存储需求。可以说，在大数据时代，云数据库将成为许多企业数据的目的地。



12.2.3 云数据库



关系数据库、NoSQL、NewSQL和云数据库产品分类

西安电子科技大学



第12章 数据存储与管理



- 12.1 传统的数据存储与管理技术
- 12.2 大数据时代的数据存储与管理技术
- 12.3 大数据处理架构Hadoop**
- 12.4 分布式文件系统HDFS
- 12.5 NoSQL数据库
- 12.6 云数据库
- 12.7 分布式数据库HBase
- 12.8 知识点小结



12.3.1 Hadoop特性

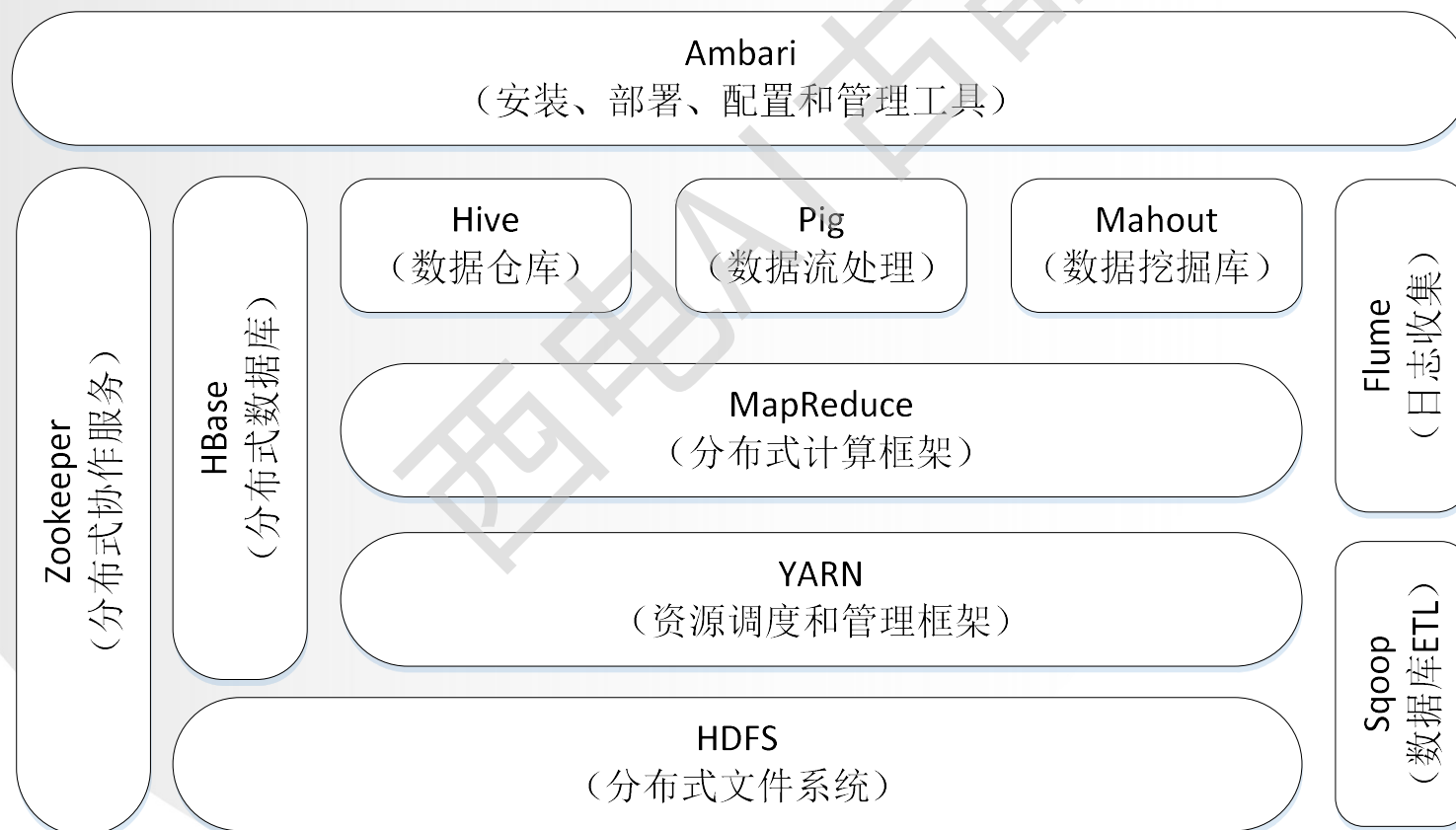
Hadoop是一个能够对大量数据进行**分布式处理**的软件框架，并且是以一种可靠、高效、可伸缩的方式进行处理的，它具有以下几个方面的特性：

- **高可靠性**
- **高效性**
- **高可扩展性**
- **高容错性**
- **成本低**
- **运行在Linux平台上**
- **支持多种编程语言**



12.3.2 Hadoop生态系统

经过多年的发展，Hadoop生态系统不断完善和成熟，目前已经包含了多个子项目。除了核心的**HDFS**和**MapReduce**以外，**Hadoop生态系统**还包括Zookeeper、HBase、Hive、Pig、Mahout、Sqoop、Flume、Ambari等功能组件。





第12章 数据存储与管理

- 12.1 传统的数据存储与管理技术
- 12.2 大数据时代的数据存储与管理技术
- 12.3 大数据处理架构Hadoop
- 12.4 分布式文件系统HDFS**
- 12.5 NoSQL数据库
- 12.6 云数据库
- 12.7 分布式数据库HBase
- 12.8 知识点小结



12.4.1 HDFS的设计目标

总体而言，**HDFS**要实现以下**目标**：

- 兼容廉价的硬件设备
- 流数据读写
- 大数据集
- 简单的文件模型
- 强大的跨平台兼容性

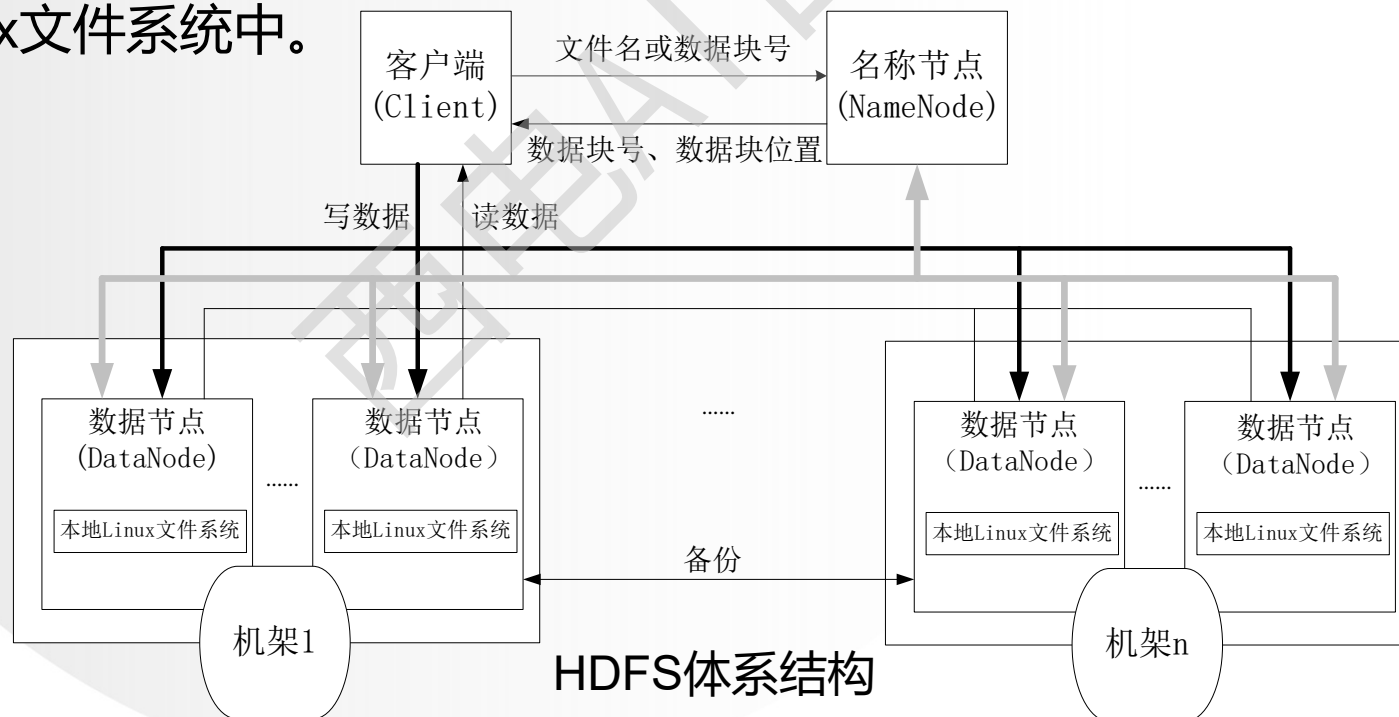
HDFS特殊的设计，在实现上述优良特性的同时，也使得自身具有一些应用**局限性**，主要包括以下几个方面：

- 不适合低延迟数据访问
- 无法高效存储大量小文件
- 不支持多用户写入及任意修改文件



12.4.2 HDFS体系结构

HDFS采用了**主从 (Master/Slave) 结构模型**，一个HDFS集群包括一个**名称节点 (NameNode)** 和若干个**数据节点 (DataNode)** (如图3-4所示)。**名称节点**作为中心服务器，负责管理文件系统的命名空间及客户端对文件的访问。集群中的**数据节点**一般是一个节点运行一个数据节点进程，负责处理文件系统客户端的读/写请求，在名称节点的统一调度下进行数据块的创建、删除和复制等操作。每个数据节点的数据实际上保存在本地Linux文件系统中。





第12章 数据存储与管理



- 12.1 传统的数据存储与管理技术
- 12.2 大数据时代的数据存储与管理技术
- 12.3 大数据处理架构Hadoop
- 12.4 分布式文件系统HDFS
- 12.5 NoSQL数据库**
- 12.6 云数据库
- 12.7 分布式数据库HBase
- 12.8 知识点小结



12.5 NoSQL数据库

□12.5.1 NoSQL概述

□12.5.2 NoSQL数据库的优势比较

□12.5.3 NoSQL数据库的类型

□12.5.4 NoSQL数据库选用原则

□12.5.5 NoSQL的CAP理论

□12.5.6 主流NoSQL数据库



12.5.1 NoSQL概述



- 随着大数据的兴起，NoSQL数据库现在成了一个极其热门的新领域。
- “**NoSQL**”不是 “No SQL”的缩写，它是 “**Not Only SQL**”的缩写。它的意义是：适用关系型数据库的时候就使用关系型数据库，**不适用的时候也没有必要非使用关系型数据库不可**，可以考虑使用更加合适的数据存储。
- 为弥补关系型数据库的不足，各种各样的NoSQL数据库应运而生。



12.5 NoSQL数据库

- 12.5.1 NoSQL概述
- **12.5.2 NoSQL数据库的优势比较**
- 12.5.3 NoSQL数据库的类型
- 12.5.4 NoSQL数据库选用原则
- 12.5.5 NoSQL的CAP理论
- 12.5.6 主流NoSQL数据库



12.5.2 关系型数据库的优势



(1) **通用性及高性能。**

(2) **突出的优势**

1>保持数据的一致性 (**事务处理**) ;

2>由于以标准化为前提, 数据更新的**开销很小** (相同的字段基本上都只有一处) ;

3>可以进行**JOIN**等复杂查询;

4>存在很多实际成果和专业技术信息 (成熟的技术) 。



12.5.2 关系型数据库的劣势

- (1) **大量数据**的写入处理存在困难;
- (2) 对有数据**更新**的表做索引或表结构(schema) 变更处理不利;
- (3) **字段不固定**时应用存在缺陷;
- (4) 对简单查询需要快速返回结果的处理**响应慢**。



12.5.2 NoSQL数据库的优势

- (1) 灵活的可**扩展性**;
- (2) 轻松应对**海量**数据;
- (3) **维护**简单;
- (4) **经济**;
- (5) **灵活**的数据模型;



12.5 NoSQL数据库

- 12.5.1 NoSQL概述
- 12.5.2 NoSQL数据库的优势比较
- **12.5.3 NoSQL数据库的类型**
- 12.5.4 NoSQL数据库选用原则
- 12.5.5 NoSQL的CAP理论
- 12.5.6 主流NoSQL数据库



12.5.3 NoSQL数据库的类型



- NoSQL的官方网站（<http://nosql-database.org>）上已经有超过225种数据库。
- 具有代表性的NoSQL数据库主要有**键值(Key/Value)存储、面向文档的数据库及面向列的数据库**三种类型。



12.5.3 键值(Key/Value)存储

- ❑ 键值(Key/Value)存储的数据库是最常见的NoSQL数据库，它的数据是以**键值**的形式存储的。
- ❑ 它的**处理速度非常快**，基本上只能通过键查询获取数据。
- ❑ 根据数据的保存方式可以分为**临时性**、**永久性**和两者兼具3种。



12.5.3 键值(Key/Value)存储



(1) 临时性: “数据有可能丢失”

Memcached属于这种类型。Memcached把**所有数据都保存在内存**中，保存和读取的速度非常快，但是当Memcached停止的时候，数据就不存在了。由于数据保存在内存中，所以无法操作超出内存容量的数据（旧数据会丢失）。有如下特点：

- 1>在内存中保存数据;
- 2>可以进行非常快速的保存和读取处理;
- 3>数据有可能丢失。



12.5.3 键值(Key/Value)存储



(2) 永久性: “数据不会丢失”

Tokyo Tyrant、**Flare**和**ROMA**等属于这种类型这里的键值存储不像Memcached那样在内存中保存数据,而是把数据保存在硬盘上。与Memcached在内存中处理数据比起来,由于必然要发生对硬盘的IO操作,所以性能上还是有差距的。但数据不会丢失是它最大的优势。有如下特点:

- 1>在硬盘上保存数据;
- 2>可以进行非常快速的保存和读取处理 (但无法与Memcached相比) ;
- 3>数据不会丢失。



12.5.3 键值(Key/Value)存储



(3) 两者兼具型

Redis属于这种类型。Redis**临时性**和**永久性**兼具，且集合了临时性键值存储和永久性键值存储的优点。Redis**首先把数据保存到内存**中，在**满足特定条件**（默认是15分钟一次以上，5分钟内10个以上，1分钟内10000个以上的键发生变更）的时候**将数据写入到硬盘**中。这样既确保了内存中数据的处理速度，又可以通过写入硬盘来保证数据的永久性。这种类型的数据库特别适合于处理数组类型的数据。有如下特点：

- 1> 同时在内存和硬盘上保存数据；
- 2> 可以进行非常快速的保存和读取处理；
- 3> 保存在硬盘上的数据不会消失（可以恢复）；
- 4> 适合于处理数组类型的数据。



12.5.3 面向文档的数据库



□ MongoDB和CouchDB属于这种类型。

(1) **不定义表结构**

特征：即使不定义表结构，也可以像定义了表结构一样使用。

(2) 可以使用**复杂的查询条件**

跟键值存储不同的是，面向文档的数据库可以通过复杂的查询条件来获取数据。这是非常容易使用的NoSQL数据库。

有如下特点：

- 1>不需要定义表结构；
- 2>可以利用复杂的查询条件。



12.5.3 面向列的数据库



□ **Cassandra**、**Hbase**和**HyperTable**属于这种类型。

(1) **面向行**的数据库和**面向列**的数据库

普通的关系型数据库（面向行的数据库）都是以行为单位来存储数据的，擅长进行以行为单位的数据处理。

面向列的数据库是**以列为单位来存储数据**的，擅长以列为单位读入数据。

(2) **高扩展性**

有如下特点：

(1) 高扩展性（特别是写入处理）：

(2) 应用十分困难。



12.5 NoSQL数据库

- 12.5.1 NoSQL概述
- 12.5.2 NoSQL数据库的优势比较
- 12.5.3 NoSQL数据库的类型
- 12.5.4 NoSQL数据库选用原则**
- 12.5.5 NoSQL的CAP理论
- 12.5.6 主流NoSQL数据库



12.5.4 NoSQL数据库选用原则

(1) 并非对立而是互补的关系

- 关系型数据库和NoSQL数据库与其说是对立的（替代关系），倒不如说是互补的。
- 通常情况下使用关系型数据库，在适合使用NoSQL的时候使用NoSQL数据库。

(2) 量材适用：“更高的处理速度”和“更恰当的数据存储”。

(3) 增加了数据存储的方式。



12.5 NoSQL数据库

- 12.5.1 NoSQL概述
- 12.5.2 NoSQL数据库的优势比较
- 12.5.3 NoSQL数据库的类型
- 12.5.4 NoSQL数据库选用原则
- 12.5.5 NoSQL的CAP理论**
- 12.5.6 主流NoSQL数据库



12.5.5 NoSQL系统是分布式系统



- **分布式系统** (Distributed System) 是建立在网络之上的软件系统，具有高度的透明性。
- **透明性**是指每一个节点对用户的应用来说都是透明的，看不出是本地还是远程。
- 在分布式数据库系统中，用户感觉不到数据是分布的，即用户不需要知道关系是否分割、有无副本、数据存于哪台机器及操作在哪台机器上执行等。
- NoSQL系统是分布式系统，因为用户是通过一些API接口来访问它们，并不知道其最终内部工作需要由很多台机器协同完成。



12.5.5 CAP理论阐述



□ 字母 “C”、“A”和 “P”分别代表了强一致性、可用性和分区容错性三个特征。

(1) **强一致性(Consistency)**

系统在执行过某项操作后仍然处于一致的状态。在分布式系统中，**更新操作**执行成功后所有的用户都应该**读取到最新的值**，这样的系统被认为具有强一致性。

(2) **可用性(Availability)**

每一个操作总是能够在一定的时间内返回结果。

(3) **分区容错性(Partition Tolerance)**

分区容错性可以理解为系统在存在**网络分区**的情况仍然可以接受请求（满足一致性和可用性）。



12.5 NoSQL数据库

- 12.5.1 NoSQL概述
- 12.5.2 NoSQL数据库的优势比较
- 12.5.3 NoSQL数据库的类型
- 12.5.4 NoSQL数据库选用原则
- 12.5.5 NoSQL的CAP理论
- 12.5.6 主流NoSQL数据库



12.5.6 主流NoSQL数据库



□ 主流NoSQL数据库

- HBase
- Redis
- MongoDB
- Couchbase
- LevelDB



12.5.6 HBase



□ HBase

- HBase 是 Apache Hadoop 中的一个子项目，属于 bigtable 的开源版本，所实现的语言为 **Java**（故依赖 Java SDK）。
- HBase 依托于 **Hadoop** 的 HDFS（分布式文件系统）作为最基本存储基础单元。
- HBase在列上实现了 BigTable 论文提到的**压缩算法**、**内存操作**和**布隆过滤器**。
- HBase的表能够作为 MapReduce任务的输入和输出，可以通过Java API来访问数据，也可以通过REST、Avro或者Thrift的API来访问。



12.5.6 HBase

□ HBase的**特点**:

(1) **数据格式**

- 数据存储是**基于列** (ColumnFamily) 的, 且非常松散。
- 允许表下某行某列值**为空时不做任何存储** (也不占位), 减少了空间占用也提高了读性能。

(2) **性能**

- **HStore存储**是HBase存储的核心, 它由两部分组成, 一部分是MemStore, 一部分是StoreFiles。

(3) **数据版本**

- 能直接检索到往昔版本的数据, 这意味着我们更新数据时, 旧数据并没有即时被清除, 而是保留着。

(4) **CAP类别**: CP类型



12.5.6 HBase



□ HBase的优缺点:

➤ **优势**

- (1) **存储容量**大，一个表可以容纳上亿行，上百万列；
- (2) 可通过版本进行检索，能搜到所需的**历史版本**数据；
- (3) 负载高时，可通过简单的添加机器来实现水平切分扩展，跟**Hadoop**的无缝集成保障了其数据可靠性（HDFS）和海量数据分析的高性能（MapReduce）；
- (4) 在第3点的基础上可有效避免**单点故障**的发生。
- (5) API相比其他 NoSql 的相对笨拙。



12.5.6 HBase



□ HBase的优缺点:

➤ 缺点

- (1) 基于**Java语言**实现及**Hadoop架构**意味着其API更适用于**Java项目**;
- (2) **node开发环境**下所需**依赖项较多**、**配置麻烦** (或不知如何配置, 如持久化配置), 缺乏文档;
- (3) **占用内存很大**, 且鉴于建立在为批量分析而优化的HDFS上, 导致读取性能不高;
- (4) API相比其他 NoSQL 的相对笨拙。



12.5.6 HBase

□ HBase的**适用场景**:

- (1) **bigtable类型**的数据存储;
- (2) 对数据有版本查询需求;
- (3) 应对超大数据量要求扩展简单的需求。



12.5.6 Redis

□ Redis

- Redis 是一个开源的使用ANSI C语言编写、支持网络、可基于内存亦可持久化的日志型、**Key-Value数据库**，并提供多种语言的API。
- 目前由VMware主持开发工作。



12.5.6 Redis

□ Redis的**特点**:

- (1) **数据格式**: Redis 通常被称为数据结构服务器, 因为值可以是**字符串**, **哈希**, **列表**, **集合**和**有序集合**五种类型。
- (2) **性能**: Redis数据库完全在内存中, 因此处理速度非常快。
- (3) **持久化**: 通过**定时快照** (snapshot) 和**基于语句的追加** (AppendOnlyFile, AOF) 两种方式, redis可以支持数据持久化。
- (4) **CAP类别**: CP类型。



12.5.6 Redis

□ Redis的优缺点:

➤ **优势**

- (1) 非常丰富的**数据结构**;
- (2) Redis提供了事务的功能,可以保证一串命令的原子性,中间不会被任何操作打断;
- (3) 数据存在内存中,读写非常的高速,可以达到10w/s的频率。



12.5.6 Redis



□ Redis的优缺点:

➤ 缺点

- (1) Redis3.0后才出来官方的集群方案，但仍存在一些架构上的问题；
- (2) 持久化功能体验不佳。
- (3) 由于是**内存数据库**，所以，单台机器存储的数据量，跟机器本身的内存大小。虽然redis本身有key过期策略，但是还是需要提前预估和节约内存。如果内存增长过快，需要定期删除数据。

➤ 适用场景

- 适用于数据变化快且数据库大小可遇见（适合内存容量）的应用程序。



12.5.6 MongoDB



□ MongoDB

- MongoDB 是一个高性能，开源，无模式的文档型数据库，**开发语言是C++**。
- 它在许多场景下可用于替代传统的**关系型数据库**或**键/值存储方式**。



12.5.6 MongoDB



□ MongoDB的**特点**

(1) **数据格式**

- 在 MongoDB 中，**文档**是对数据的抽象，它的表现形式是BSON（Binary JSON）。
- BSON 是一个轻量级的**二进制数据格式**。MongoDB 能够使用 BSON，并将 BSON 作为数据的存储存放在磁盘中。

(2) **性能**

- MongoDB 目前支持的存储引擎为内存映射引擎。
- 引擎的特点：
 - MongoDB 中关于内存管理的代码非常精简；
 - MongoDB 服务器使用的虚拟内存将非常巨大，并将超过整个数据文件的大小。



12.5.6 MongoDB



□ MongoDB的**特点**

(2) **性能**

- MongoDB 提供了**全索引支持**：包括文档内嵌对象及数组。Mongo的查询优化器会分析查询表达式，并生成一个高效的查询计划。通常能够极大的提高查询的效率。

(3) **持久化**

MongoDB 在1.8版本之后开始支持 journal，就是我们常说的 redo log，用于故障恢复和持久化。

(5) **CAP类别**

MongoDB 比较灵活，可以设置成 strong consistent (**CP类型**) 或者 eventual consistent (**AP类型**)。默认是 CP 类型。



12.5.6 MongoDB



□ MongoDB的优缺点

➤ 优势

- (1) 强大的自动化 shading 功能;
- (2) 全索引支持, **查询非常高效**;
- (3) 面向**文档 (BSON) 存储**, 数据模式简单而强大;
- (4) 支持**动态查询**, 查询指令也使用JSON形式的标记, 可轻易查询文档中内嵌的对象及数组;
- (5) **支持 javascript 表达式查询**, 可在服务器端执行任意的 javascript函数。



12.5.6 MongoDB



□ MongoDB的优缺点

➤ 缺点

- (1) **单个文档大小限制**为16M，32位系统上，不支持大于2.5G的数据；
- (2) 对**内存要求比较大**，至少要保证热数据（索引，数据及系统其他开销）都能装进内存；
- (3) **非事务机制**，无法保证事件的原子性。



12.5.6 MongoDB



□ MongoDB的**适用场景**

- (1) 适用于实时的插入、更新与查询的需求，并具备应用程序实时数据存储所需的复制及高度伸缩性；
- (2) 非常适合**文档化格式**的存储及查询；
- (3) **高伸缩性**的场景：MongoDB 非常适合由数十或者数百台服务器组成的数据库。
- (4) 对性能的关注超过对功能的要求。

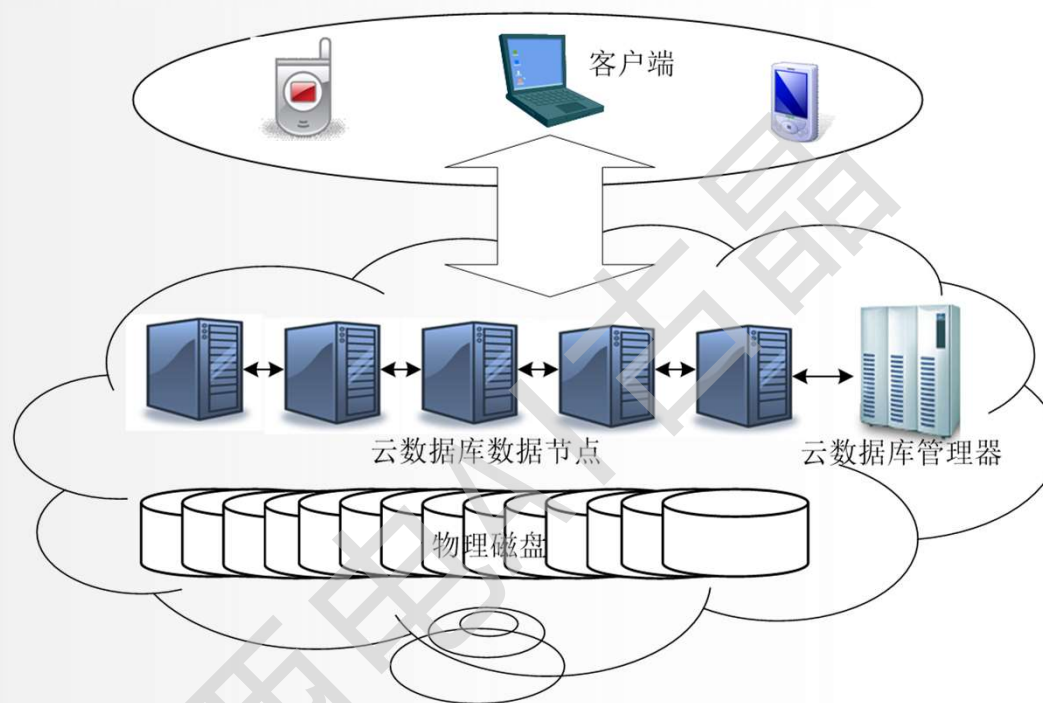


第12章 数据存储与管理

- 12.1 传统的数据存储与管理技术
- 12.2 大数据时代的数据存储与管理技术
- 12.3 大数据处理架构Hadoop
- 12.4 分布式文件系统HDFS
- 12.5 NoSQL数据库
- 12.6 云数据库**
- 12.7 分布式数据库HBase
- 12.8 知识点小结



12.6.1 云数据库的概念



云数据库是部署和虚拟化在云计算环境中的数据库。云数据库是在云计算的大背景下发展起来的一种新兴的**共享基础架构**的方法，它极大地增强了数据库的存储能力，消除了人员、硬件、软件的重复配置，让软、硬件升级变得更加容易。云数据库具有**高可扩展性**、**高可用性**、采用**多租形式**和**支持资源有效分发**等特点。



12.6.2 云数据库的特性

腾讯云数据库和自建数据库的比较

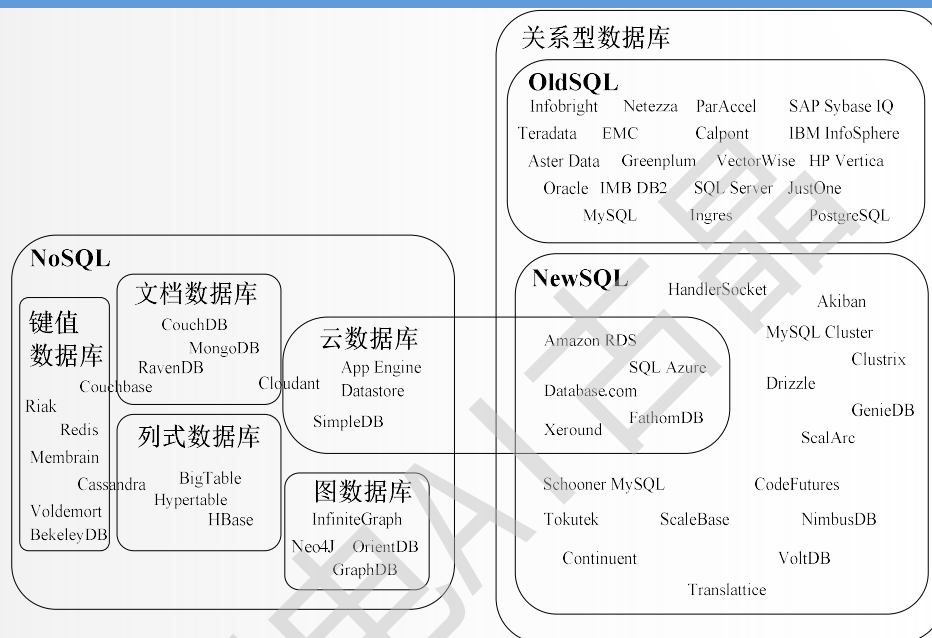
云数据库具有以下**特性**:

- (1) 动态可扩展
- (2) 高可用性
- (3) 较低的使用代价
- (4) 易用性
- (5) 高性能
- (6) 免维护
- (7) 安全

	自建数据库	腾讯云数据库
数据安全性	开发者自行解决, 成本高昂	15种类型备份数据, 保证数据安全
服务可用性		99.99%高可靠性
数据备份		0花费, 系统自动多时间点数据备份
维护成本		0成本, 专业团队7x24小时帮助维护
实例扩容		一键式直接扩容, 安全可靠
资源利用率		按需申请, 资源利用率高达99.9%
技术支持		专业团队一对一指导、QQ远程协助开发者



12.6.3 云数据库与其他数据库的关系



- 从数据模型的角度来说，云数据库并非一种全新的数据库技术，而只是以**服务**的方式提供数据库功能。
- 云数据库**并没有专属于自己的数据模型**，云数据库所采用的数据模型可以是关系数据库所使用的**关系模型**（微软的SQL Azure云数据库、阿里云RDS都采用了关系模型），也可以是NoSQL数据库所使用的**非关系模型**（Amazon Dynamo云数据库采用的是“键/值”存储）。
- 同一个公司也可能提供采用不同数据模型的多种云数据库服务。
- 许多公司在开发云数据库时，后端数据库都是直接使用现有的各种关系数据库或NoSQL数据库产品。



12.6.4代表性云数据库产品

云数据库产品

企业	产品
Amazon	Dynamo、SimpleDB、RDS
Google	Google Cloud SQL
Microsoft	Microsoft SQL Azure
Oracle	Oracle Cloud
Yahoo!	PNUTS
Vertica	Analytic Database v3.0 for the Cloud
EnterpriseDB	Postgres Plus in the Cloud
阿里	阿里云RDS
百度	百度云数据库
腾讯	腾讯云数据库



第12章 数据存储与管理

- 12.1 传统的数据存储与管理技术
- 12.2 大数据时代的数据存储与管理技术
- 12.3 大数据处理架构Hadoop
- 12.4 分布式文件系统HDFS
- 12.5 NoSQL数据库
- 12.6 云数据库
- 12.7 分布式数据库HBase**
- 12.8 知识点小结



12.7.1 BigTable



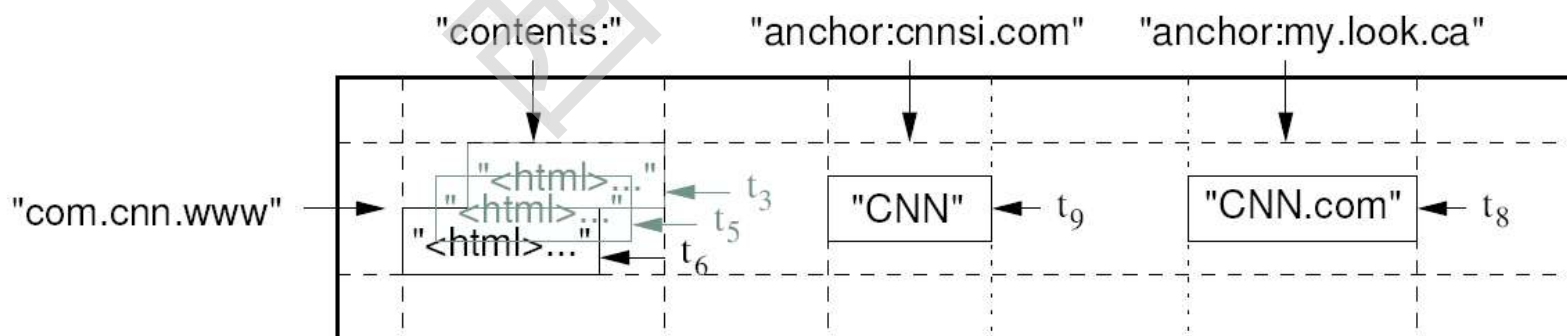
BigTable是一个**分布式存储系统**，起初用于解决典型的互联网搜索问题。

➤ 建立互联网索引

- (1) 爬虫持续不断地抓取新页面，这些页面每页一行地存储到BigTable里。
- (2) MapReduce计算作业运行在整张表上，生成索引，为网络搜索应用做准备。

➤ 搜索互联网

- (3) 用户发起网络搜索请求。
- (4) 网络搜索应用查询建立好的索引，从BigTable得到网页。
- (5) 搜索结果提交给用户。



网页在BigTable中的存储样例



12.7.1 BigTable

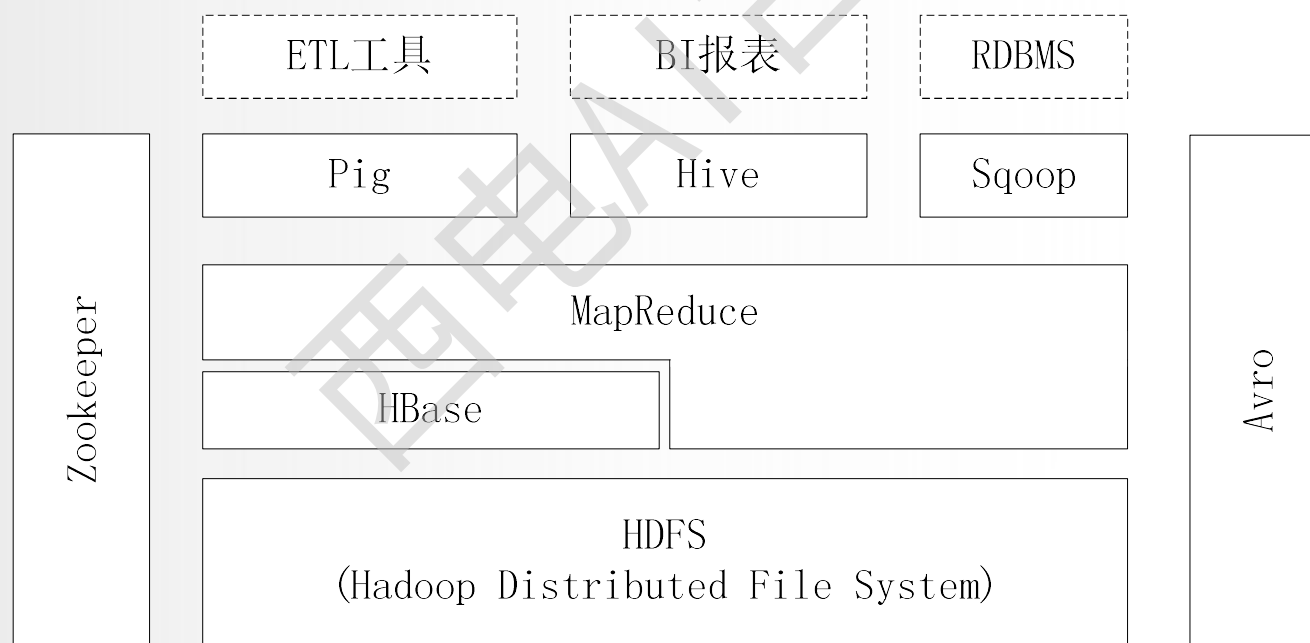
- BigTable是一个分布式存储系统。
- 利用谷歌提出的**MapReduce分布式并行计算模型**来处理海量数据。
- 使用**谷歌分布式文件系统GFS**作为底层数据存储。
- 采用Chubby提供**协同服务管理**。
- 可以扩展到PB级别的数据和上千台机器，具备**广泛应用性、可扩展性、高性能和高可用性**等特点。
- 谷歌的许多项目都存储在BigTable中，包括搜索、地图、财经、打印、社交网站Orkut、视频共享网站YouTube和博客网站Blogger等。



12.7.2 HBase简介



HBase是一个高可靠、高性能、面向列、可伸缩的分布式数据库，是谷歌BigTable的开源实现，**主要用来存储非结构化和半结构化的松散数据**。HBase的目标是处理非常庞大的表，可以通过水平扩展的方式，利用廉价计算机集群处理由超过10亿行数据和数百万列元素组成的数据表。



Hadoop生态系统中HBase与其他部分的关系



12.7.3 HBase数据模型



- **表**: HBase采用表来组织数据, 表由行和列组成, 列划分为若干个列族。
- **行**: 每个HBase表都由若干行组成, 每个行由行键 (row key) 来标识。
- **列族**: 一个HBase表被分组成许多“列族” (Column Family) 的集合, 它是基本的访问控制单元。
- **列限定符**: 列族里的数据通过列限定符 (或列) 来定位。
- **单元格**: 在HBase表中, 通过行、列族和列限定符确定一个“单元格” (cell), 单元格中存储的数据没有数据类型, 总被视为字节数组byte[]。
- **时间戳**: 每个单元格都保存着同一份数据的多个版本, 这些版本采用时间戳进行索引。

	Info		
	name	major	email
201505001	Luo Min	Math	luo@qq.com
201505002	Liu Jun	Math	liu@qq.com
201505003	Xie You	Math	xie@qq.com you@163.com

列限定符

列族

行键

单元格

ts1

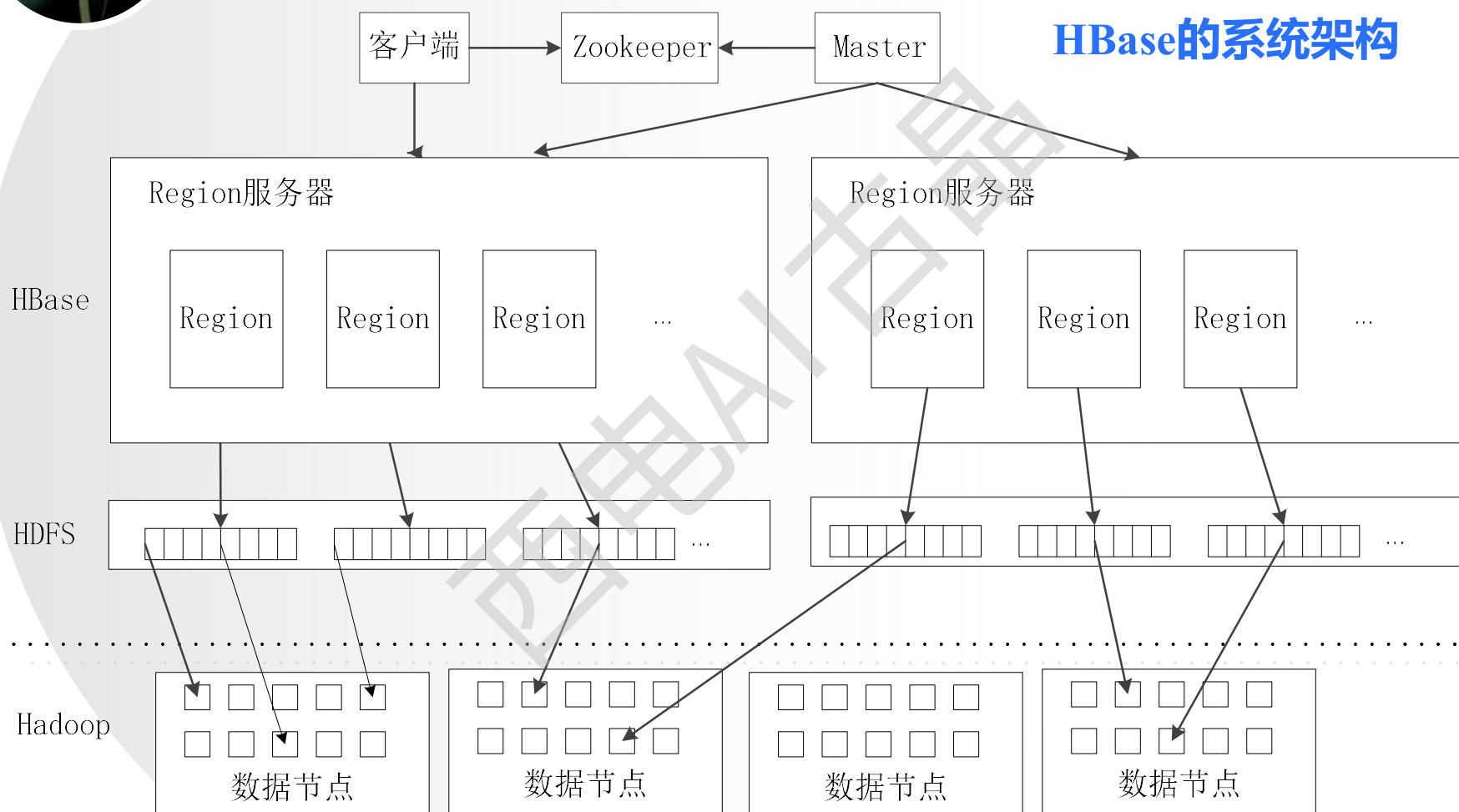
ts2

该单元格有2个时间戳ts1和ts2
每个时间戳对应一个数据版本
ts1=1174184619081 ts2=1174184620720



12.7.4 HBase系统架构

HBase的系统架构





第12章 数据存储与管理



- 12.1 传统的数据存储与管理技术
- 12.2 大数据时代的数据存储与管理技术
- 12.3 大数据处理架构Hadoop
- 12.4 分布式文件系统HDFS
- 12.5 NoSQL数据库
- 12.6 云数据库
- 12.7 分布式数据库HBase
- 12.8 知识点小结**



12.8 知识点小结



本章知识小结:

- NoSQL数据库的优缺点与类型
- 主流NoSQL数据库
- 分布式数据库HBase的特点与优缺点



Thanks!
See you